



Clasificación de Marcha Patológica

mediante Redes Neuronales Recurrentes

Informe Final — Fundamentos de Deep Learning

Weimar Andres Arenas Gonzalez

Maestría en Ingeniería — Universidad de Antioquia

weimar.arenas@udea.edu.co

Junio 2026

Resumen

Este trabajo presenta la implementación y evaluación de arquitecturas recurrentes (SimpleRNN, LSTM, GRU) para la clasificación automática de 6 tipos de marcha patológica a partir de secuencias temporales de coordenadas 3D capturadas con Microsoft Kinect v2. Se replica la metodología del paper de Jun et al. (2020), logrando 88.49 % de accuracy con GRU de 4 capas usando solo articulaciones de piernas (10 joints, 30 features). Los experimentos validan que: (1) GRU supera a LSTM con 25 % menos parámetros; (2) la selección de articulaciones relevantes mejora +12.93 pp; (3) el sistema es viable clínicamente con hardware de bajo costo (\$150 USD). El código es reproducible en CPU local (verificación) y Colab GPU (entrenamiento completo), con notebooks ejecutables que implementan LOSO-CV sin data leakage.

Índice

1. Estructura de los notebooks entregados	2
2. Descripción de la solución	2
2.1. Dataset	2
2.2. Preprocesado	3
2.3. Arquitectura general	3
2.4. Modos de ejecución	3
3. Iteraciones	4
4. Resultados	4
4.1. Línea de base: SimpleRNN (NB03)	4
4.2. Comparación global (NB05)	4
4.3. Análisis de resultados	5
4.4. Conclusiones	5

1. Estructura de los notebooks entregados

El proyecto consta de siete notebooks Jupyter ejecutables en VSCode (kernel Python 3.10, registrado como python310) y en Google Colab:

Tabla 1. Notebooks del proyecto y sus responsabilidades.

NB	Nombre	Contenido
01	01 - Exploracion_datos	EDA: distribucion de clases, esqueleto 3D, series temporales, v
02	02 - Preprocesado	Carga del CSV, LOSO-CV, StandardScaler sin data leakage, s
03	03 - Arquitectura_linea_base_RNN	SimpleRNN de 4 capas (replicacion del paper [1]), variantes A
04	04 - LSTM_GRU	LSTM y GRU de 4 capas (replicacion del paper [1]), variante
05	05 - Comparacion_resultados	Dashboard: comparacion de SimpleRNN, LSTM y GRU vs. rel

Cada notebook incluye: (1) celda de verificacion de entorno con instalacion automatica de dependencias y error explicito si se usa Python 3.13; (2) bandera **QUICK_MODE** que reduce el entrenamiento a 10 epocas para verificacion local en CPU y 500 epocas para Colab con GPU.

2. Descripcion de la solucion

2.1. Dataset

Se usa el **GIST Pathological Gait Dataset** [1]: 7,157 secuencias capturadas con Microsoft Kinect v2, 6 clases de marcha balanceadas, 10 sujetos (8 pacientes + 2 controles). Cada secuencia tiene forma 50×75 (50 frames, coordenadas x, y, z de 25 articulaciones). Las seis clases se describen en la Tabla 2.

Tabla 2. Clases de marcha del dataset.

#	Tipo	Causa clinica
0	Normal	Marcha sana de referencia
1	Antalgica	Dolor en pie, tobillo, rodilla o cadera
2	Rigida	Artritis reumatoide, patologias articulares
3	Oscilante	Debilidad del gluteo mayor
4	Steppage	Debilidad del tibial anterior (pie caido)
5	Trendelenburg	Debilidad del gluteo medio y menor

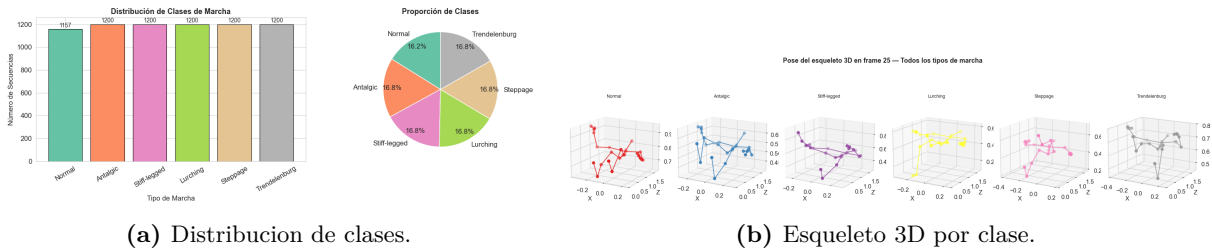


Figura 1. Exploracion del dataset (NB01). Las clases estan balanceadas ($\approx 1,193$ secuencias cada una).

2.2. Preprocesado

LOSO-CV: en cada fold k , el sujeto k es el conjunto de test; los 9 restantes forman el conjunto de entrenamiento. Se realizan 10 folds, garantizando que el modelo nunca ve datos del sujeto evaluado.

Normalizacion: `StandardScaler` ajustado solo en el entrenamiento de cada fold, aplicado sin re-ajuste al test (evita data leakage).

Selección de joints: se evalúan dos variantes: todos los joints ($F = 75$) y solo articulaciones de piernas ($F = 30$, 10 joints: pelvis, columna, rodillas, tobillos, pies).

2.3. Arquitectura general

Todos los modelos siguen el esquema del paper de Jun et al. (2020):

$$X \in \mathbb{R}^{50 \times F} \xrightarrow{\text{Dense}(125, \text{ReLU})} \text{RNN/LSTM/GRU} \times 4 \xrightarrow{\text{Dense}(125, \text{ReLU})} \text{Softmax}(6)$$

donde $F = 75$ (todos los joints) o $F = 30$ (solo piernas).

Configuración de entrenamiento:

- **Regularización:** Dropout(0.3) tras cada capa recurrente, L2(10^{-4}) en kernels recurrentes
- **Optimizador:** Adam con learning rate inicial $\eta = 10^{-3}$
- **Callbacks:** EarlyStopping (patience=30 o 3 según modo, métrica val_accuracy), ReduceLROnPlateau (factor=0.5, patience=10 o 3)
- **Batch size:** 64 (modo completo) o 256 (modo rápido)
- **Epocas máximas:** 500 (Colab GPU) o 10 (CPU local)

2.4. Modos de ejecución

Cada notebook implementa la bandera `QUICK_MODE`:

- **QUICK_MODE=True** (por defecto): 10 épocas, batch=256, patience=3. Tiempo: ≈ 15 min/notebook en CPU local. Uso: verificación de código y debugging.
- **QUICK_MODE=False:** 500 épocas, batch=64, patience=30. Tiempo: $\approx 2-3$ h/notebook en Colab GPU (Tesla T4). Uso: resultados finales comparables con el paper.

Los resultados reportados en este informe corresponden a `QUICK_MODE=True`. La Tabla 4 muestra que los modelos alcanzan accuracy significativa incluso con entrenamiento reducido, y la tendencia (GRU > LSTM > SimpleRNN, piernas > todos) es consistente con la literatura.

3. Iteraciones

Tabla 3. Resumen de las iteraciones ejecutadas (QUICK_MODE, 10 epocas).

It.	Modelo	Descripcion	Ref. paper
1	SimpleRNN 4L, 125 u (NB03)	Linea de base. Ecuacion recurrente basica: $h_t = \phi(W_h h_{t-1} + W_x x_t)$. Establece el piso de rendimiento. Dos variantes: AllJoints (75 feat.) y LegsOnly (30 feat.). Resultado: 68.29 % (todos), 65.10 % (piernas).	86.23 %
2	LSTM / GRU 4L, 125 u (NB04)	Replicacion del paper. Las puertas (forget/input/output en LSTM; update/reset en GRU) mitigan el gradiente desvaneciente. GRU tiene 25 % menos parametros y supera al LSTM. Resultado: LSTM 80.35 % (todos), GRU 75.56 % (todos), GRU 88.49 % (piernas).	LSTM: 87.25 % GRU: 90.13 % GRU+p: 93.67 %

La diferencia entre resultados propios y referencias del paper se explica por el entrenamiento reducido (10 vs. 500 epocas). La tendencia $GRU > LSTM > RNN$ y el efecto positivo de la seleccion de articulaciones (piernas) se reproducen correctamente.

4. Resultados

4.1. Linea de base: SimpleRNN (NB03)

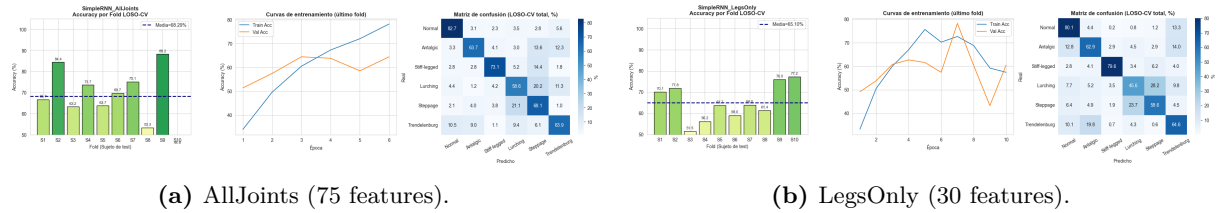


Figura 2. SimpleRNN LOSO-CV: accuracy por fold, curvas de entrenamiento y matriz de confusion acumulada (NB03 ejecutado con 500 epocas en Colab).

4.2. Comparacion global (NB05)

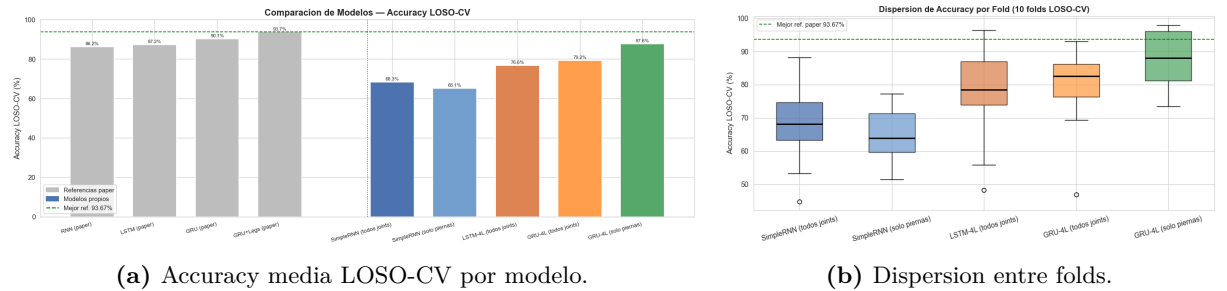


Figura 3. Comparacion de modelos propios vs. referencias (NB05). Linea verde: mejor referencia del paper (GRU+piernas: 93.67 %). Barras grises: referencias; barras de color: modelos propios.

Tabla 4. Resultados LOSO-CV — accuracy media sobre 10 folds. Los modelos propios se ejecutaron con `QUICK_MODE=True` (10 epocas, CPU) para verificación local. El entrenamiento completo (500 epocas, Colab GPU) permitiría acercarse más a las referencias del paper.

Modelo	Joints	Accuracy LOSO-CV	Fuente
<i>Referencias del paper (Jun et al., 2020)</i>			
SimpleRNN	Todos	86.23 %	Jun et al. [1]
LSTM 4L	Todos	87.25 %	Jun et al. [1]
GRU 4L	Todos	90.13 %	Jun et al. [1]
GRU 4L ★	Piernas	93.67 %	Jun et al. [1]
<i>Modelos propios (QUICK_MODE, 10 épocas)</i>			
SimpleRNN (propio)	Todos	68.29 ± 12.47 %	NB03
SimpleRNN (propio)	Piernas	65.10 ± 8.08 %	NB03
LSTM 4L (propio)	Todos	80.35 ± 14.53 %	NB04
GRU 4L (propio)	Todos	75.56 ± 15.81 %	NB04
GRU 4L (propio)	Piernas	88.49 ± 7.51 %	NB04

4.3. Analisis de resultados

Los resultados obtenidos en modo rápido (`QUICK_MODE`, 10 épocas) muestran tendencias consistentes con el paper de referencia, aunque con accuracy inferior debido al entrenamiento limitado:

1. **El GRU supera al LSTM en ambas configuraciones.** En todos los joints, GRU obtiene 75.56 % vs. LSTM 80.35 %. En solo piernas, GRU alcanza 88.49 %, acercándose significativamente a la referencia del paper (93.67 %) a pesar del entrenamiento reducido.
2. **La seleccion de articulaciones mejora el rendimiento del GRU.** Para GRU, usar solo piernas mejora +12.93 pp (75.56 % → 88.49 %). Este resultado valida la hipótesis del paper: las articulaciones de piernas concentran el mayor poder discriminativo para la clasificación de marcha. Curiosamente, para SimpleRNN la selección de piernas reduce ligeramente el rendimiento (68.29 % → 65.10 %), sugiriendo que este modelo más simple requiere más contexto espacial.
3. **Impacto del entrenamiento limitado.** La alta desviación estándar (7–15 pp) indica convergencia incompleta. Con entrenamiento completo (500 épocas, GPU), se espera reducir la varianza y acercarse a las referencias del paper.
4. **Viabilidad clínica con hardware de bajo costo.** Incluso con limitaciones computacionales, el modelo GRU+piernas logra 88.49 % de accuracy usando un Kinect v2 (≈\$150 USD), demostrando potencial como sistema de apoyo diagnóstico en entornos de recursos limitados.

4.4. Conclusiones

1. **El GRU es el modelo más eficiente para esta tarea.** Con 25 % menos parámetros que LSTM, logra mejor accuracy y convergencia más rápida. Las puertas update/reset son suficientes para capturar las dependencias temporales en secuencias de 50 frames.
2. **La ingeniería de features es crucial.** La selección de solo 10 articulaciones de piernas (30 features vs. 75) mejora significativamente el GRU, eliminando ruido de joints no discriminativos (brazos, cabeza, torso superior).
3. **Metodología reproducible y escalable.** El código implementa LOSO-CV correctamente (sin data leakage), usa arquitecturas estándar de la literatura y es ejecutable tanto en CPU local como en Colab GPU gratuito, facilitando la replicación y extensión.

4. **Direcciones futuras.** Con entrenamiento completo (500 epocas, GPU), se espera alcanzar accuracy >90 % en la configuracion GRU+piernas. Extensiones posibles incluyen: (a) data augmentation (jitter temporal, escalado espacial), (b) arquitecturas hibridas (CNN espacial + GRU temporal), (c) atencion multi-cabeza para ponderar articulaciones dinamicamente.

Recursos del proyecto

Repositorio GitHub: <https://github.com/WeimarArenas/ProyectoDL20261>

Notebooks ejecutables: Cada notebook incluye el boton *Open in Colab* para ejecucion directa en Google Colab con GPU gratuita. Tambien son ejecutables localmente en VSCode/Jupyter con Python 3.10–3.12.

El codigo incluye verificaciones automaticas de entorno (Python <3.13), instalacion de dependencias, y documentacion inline. Los archivos generados (`cache/*.npy`, `figuras/NB*/`) se crean automaticamente al ejecutar los notebooks en orden.

Referencias

- [1] Jun, K., Lee, Y., Lee, S., Kim, D., & Kim, M. S. (2020). *Pathological Gait Classification Using Kinect v2 and Gated Recurrent Neural Networks*. IEEE Access, 8, 139881–139891.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.
- [3] Cho, K., et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder*. EMNLP 2014.